

Matricule N°: _____ - _____ - _____.

Nom : _____.

Prénom : _____.

Programme : _____.

GIF-1003– Programmation avancée en C++

Examen final Automne 2020

Durée: 2h50

Aucun document autorisé

Lisez l'examen en entier avant de le commencer. Attention, chaque détail compte.

- Ce questionnaire comporte 4 parties sur 14 pages (incluant une annexe).
- Veuillez répondre dans les espaces prévus sur le questionnaire.
- Soyez concis dans vos réponses, les espaces devant être suffisants pour contenir vos réponses. Vous pouvez vous servir du verso comme brouillon.

1. Principes (12 points)

Théorie du contrat (4 points)

Jean utilise une classe développée par **Rémi** dans son projet. Situez les responsabilités si **Jean** compile son programme et trouve les erreurs suivantes:

1. PRECONDITIONEXCEPTION (☐ Jean ☐ Rémi).
2. POSTCONDITIONEXCEPTION (☐ Jean ☐ Rémi).
3. INVARIANTEXCEPTION (☐ Jean ☐ Rémi).

À quoi correspond l'invariant d'une classe?

a) Des conditions qui précisent ce qui ne peut pas varier dans une classe	b) Des conditions qui précisent les contraintes de validité à respecter au moment de la construction des objets par la classe	c) Des conditions qui précisent les contraintes de validité des objets devant être respectées pendant toute leur durée de vie de leur construction à leur destruction	d) Des contraintes de validité qui doivent être vérifiées dans toutes les méthodes de la classe
---	---	---	---

(Indiquer la ou les bonnes réponses) _____

Gestion mémoire (4 points)

Dans quels cas de figure le constructeur copie est-il appelé implicitement ?

Pour la correction	p1	
	p2	
	p3	
	p4	
	p5	
	p6	
	p7	
	p8	
	p9	
	p10	
	p11	
	Total	

De ce qui suit, qu'est-ce qui est automatiquement ajouté à toute classe si on ne l'implémente pas

a) le constructeur de copie	b) l'opérateur d'assignation	c) un constructeur sans paramètre
-----------------------------	------------------------------	-----------------------------------

(Indiquer la ou les bonnes réponses) _____

Polymorphisme (4 points)

Par quels moyens et comment le liage dynamique (polymorphisme) est-il réalisé ?

2. Application

Nous voulons développer un programme afin de gérer l'inventaire d'une épicerie. Ce programme est construit au fur et à mesure. Sa construction est guidée par une série de questions indépendantes.

Dans tout le développement, la théorie du contrat doit être appliquée (il ne sera pas toujours précisé explicitement dans les questions qui suivent qu'il faut le faire. Alors, pensez-y systématiquement.

Pour cela vous disposez des macros définies dans le module `ContratException` (fichiers `ContratException.h` et `ContratException.cpp`):

`PRECONDITION(test logique)`

`INVARIANT(test logique)`

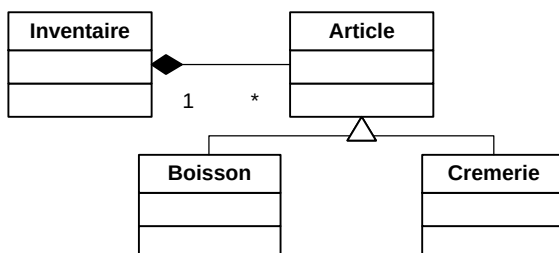
`POSTCONDITION(test logique)`

`INVARIANTS()`

`INVARIANTS()` appelle la méthode `verifieInvariant` que vous allez devoir implémenter.

Hérarchie de classe, Héritage et polymorphisme

Les données manipulées correspondent à des articles de différentes catégories. Voici la hiérarchie de classe utilisée :



ces classes seront implémentées dans l'espace de nom

examenFinal

Toutes les interfaces de ces classes sont données, mais ne sont pas tout à fait complètes. **Il faut les compléter au fur et à mesure si besoin.** Certaines méthodes sont déclarées dans ces interfaces, mais ne sont pas à implémenter, supposant qu'elles le sont déjà. Pensez à les utiliser au besoin.

Interface de la classe Article (6 points):

```
// Fichier : Article.h
```

```
#ifndef ARTICLE_H_DEJA_INCLU
```

```
#define ARTICLE_H_DEJA_INCLU
```

```
#include <string>
```

```
class Article
```

```
{
```

```
    ~Article    () {};
```

```
    Article*    clone    ()        ;
```

```
    std::string reqArticleFormate ()        ;
```

```
    reqReference ()        ;//accesseur
```

```
    reqQuantite  ()        ;//accesseur
```

```
    std::string  m_reference;
```

```
    int          m_quantite;
```

```
};
```

```
#endif
```

Interface de la classe Boisson (6 points):

```
// Fichier : Boisson.h
```

```
#ifndef BOISSON_H_DEJA_INCLU
```

```
#define BOISSON_H_DEJA_INCLU
```

```
class Boisson
```

```
{
```

```
    Article*    clone    ()        ;
```

```
    std::string reqArticleFormate ()        ;
```

```
    float       reqContenance  ()        ;//accesseur
```

```
    float       m_contenance;
```

```
};
```

```
#endif
```

Interface de la classe Inventaire (11 points):

```
// Fichier Inventaire.h
```

```
#ifndef INVENTAIRE_H_DEJA_INCLU
```

```
#define INVENTAIRE_H_DEJA_INCLU
```

```
#include <vector>
```

```
#include "ContratException.h"
```

```
#include "Date.h"
```

```
class Inventaire
```

```
{
```

```
    Inventaire(const util::Date& uneDate);
```

```
    reqDate      ()          ;//accesseur
```

```
    std::string  reqInventaireFormate ()          ;
```

```
    util::Date   m_date; // date de l'inventaire
```

```
};
```

```
#endif
```

Pour la classe `Article` (9 points)

On vous demande d'écrire les fonctions membres suivantes :

- un constructeur avec paramètres. Les paramètres sont la référence qui doit commencer par 3 lettres et être composée d'au moins 8 caractères et la quantité (pensez au contrat). Prévoir ici l'en-tête de commentaires de spécification qui seront utilisés pour générer la documentation avec Doxygen (voir l'annexe pour les balises). Cet en-tête doit être complet.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Les commentaires de spécification ne sont pas demandés dans ce qui suit.

- La fonction `reqArticleFormate` qui doit être implantée **systématiquement** dans toute classe dérivée de la classe `Article` pour permettre le polymorphisme. Pensez à faire une déclaration en conséquence. Cette méthode retourne un objet `string` avec mise en forme ayant le format suivant :

liqueur256
50

Pour un article de référence `liqueur256` en quantité de 50 (pensez à utiliser une `ostream`. La méthode `str` appliquée sur une `ostream` retourne un objet `string`)
Il n'est pas demandé de la définir ici, supposant qu'elle l'est déjà.

La fonction `verifieInvariant` pour pouvoir implanter le contrat

Complétez `Article.h` (pensez aussi à compléter les déclarations des deux accesseurs, vous n'avez cependant pas à écrire leur implémentation)

La classe `Article` est une classe abstraite (voir plus loin).

Pour la classe `Boisson` (12 points)

La classe `Boisson` est dérivée de la classe `Article` tel que présenté plus haut. On vous demande d'écrire les fonctions membres suivantes :

- un constructeur avec paramètres. Les paramètres sont une référence, la quantité en stock et la contenance en litres qui doit être positive, mais inférieure à 5.

La fonction `verifieInvariant` pour pouvoir implanter le contrat

- La méthode `clone` qui doit être implantée **systematiquement** dans toute classe dérivée de la classe `Article`, permet de faire une copie sur le monceau (tas ou heap) de l'objet courant. Ceci permet d'ajouter des `Articles` dans l'inventaire tout en autorisant le polymorphisme (dans la classe `Inventaire`)

- La méthode `reqArticleFormate` qui retourne un objet `string` avec mise en forme ayant le format suivant :

Boisson liqueur256 50 1

Pour un article `Boisson` (nom de la classe), de référence `liqueur256`, en quantité de 50 et de contenance 1litre
(pensez à utiliser une `ostream`. La méthode `str` appliquée sur une `ostream` retourne un objet `string`)

Pour la classe `Inventaire` (29 points):

La classe `Inventaire` comprend deux attributs; la date de l'inventaire et un `vector` contenant des pointeurs vers des `Articles` (qui peuvent être des objets `Boisson`, `Cremerie`, ...) de l'inventaire. On vous demande d'écrire les fonctions membres suivantes :

- une méthode `ajouterArticle`, qui permet d'ajouter des `Articles` à l'inventaire. La méthode `push_back` appliquée sur un `vector` permet d'ajouter un élément passé en paramètre à la fin du `vector` seulement si cet article n'est pas déjà présent dans celui-ci. La méthode "interne à la classe" `articleEstDejaPresent` permet de le vérifier (à implémenter ci-dessous). Dans le cas où l'inventaire contient déjà cet article, une exception du type `ArticleDejaPresentException` sera lancée. La classe `Inventaire` est responsable de la gestion de la mémoire.

ajouterArticle (

- La méthode `articleEstDejaPresent`
`articleEstDejaPresent (const std::string& p_reference)`

- un destructeur.

- une méthode `reqInventaireFormate` qui retourne un objet string formaté (voir le prototype dans l'interface). **Utiliser ici un iterator.** Le format de l'objet string doit être le suivant :

(Pensez à utiliser une ostream, la méthode reqDateFormatee appliquée sur un objet Date retourne celui-ci sous la forme d'un objet string formaté comme suit : Dimanche 1e 26 novembre 2017)

- Un constructeur de copie

Rappel : Avez-vous pensé à mettre à jour les interfaces ?

3. Classe ArticleDejaPresentException (3 points)

Cette classe permet de gérer l'exception de l'ajout d'un doublon d'article dans l'inventaire. Cette classe hérite de `std::runtime_error` parce que cette exception est une erreur qui pourra toujours se produire, ce n'est pas une erreur de programmation.

Elle contient seulement une méthode qui est le constructeur suivant :

```
ArticleDejaPresentException (const std::string& p_raison);
```

Ce constructeur ne fait qu'appeler le constructeur de la classe parent en lui passant la raison comme paramètre. La méthode `what()` de la classe `std::exception` au haut de la hiérarchie permet d'obtenir la string qu'on a passé.

On vous demande de compléter l'interface de cette classe

```
// Fichier ArticleDejaPresentException.h
```

```
#include <stdexcept>
```

```
class ArticleDejaPresentException
```

```
{
```

et d'écrire l'implémentation du constructeur.

```
ArticleDejaPresentException (const std::string& p_raison)
```

4. Test unitaire (12 points)

On vous demande d'implémenter le test unitaire du constructeur de la classe Article, ceci en utilisant les fonctionnalités offertes par le framework GoogleTest.

Les principales macros et assertions sont rappelées en annexe.

```
/**
 * \file ArticleTesteur.cpp
 *  Implantation des tests unitaires de la classe Article
 */
#include <gtest/gtest.h>
#include "Article.h"

using namespace std;

using namespace examenFinal;
```

[illegible]

Annexe

```
size_t size() const;
```

Return length of string

Returns the length of the string, in terms of number of characters.

This is the number of actual characters that conform the contents of the [string](#), which is not necessarily equal to its storage [capacity](#).

Both `string::size` and [string::length](#) are synonyms and return the same value.

Parameters

None

Return Value

The number of characters in the string.

[size_t](#) is an unsigned integral type.

```
int isalnum ( char c );
```

Checks whether *c* is either a decimal digit or an uppercase or lowercase letter.

Parameters

c

Character to be checked

Return Value

A value different from zero (i.e., `true`) if indeed *c* is either a digit or a letter. Zero (i.e., `false`) otherwise.

```
int isalpha ( char c );
```

Checks whether *c* is an alphabetic letter.

Parameters

c

Character to be checked

Return Value

A value different from zero (i.e., `true`) if indeed *c* is an alphabetic letter. Zero (i.e., `false`) otherwise.

```
int isdigit ( char c );
```

Checks whether *c* is a decimal digit character.

Decimal digits are any of: 0 1 2 3 4 5 6 7 8 9

Parameters

c

Character to be checked

Return Value

A value different from zero (i.e., `true`) if indeed *c* is a decimal digit. Zero (i.e., `false`) otherwise.

`ostreamstream` fournit une interface pour manipuler des chaînes de caractères comme des flux de sortie.

membres public :

```
string str ( ) const; pour obtenir l'objet string associé (fonction membre publique)
```

La méthode `size()` appliquée sur un vector retourne le nombre d'éléments qu'il contient.

La méthode `begin()` appliquée sur un vector retourne un iterator pointant sur le début de celui-ci.

La méthode `end()` appliquée sur un vector retourne un iterator pointant un élément passé la fin de celui-ci.

Balises Doxygen

```
/** ... Documentation ... */  
\file  
Permet de créer un bloc de documentation pour un fichier source ou d'en-tête.  
\brief  
Permet de créer une courte description dans un bloc de documentation. La description  
peut se faire sur une seule ligne ou plusieurs.  
\author  
Permet d'ajouter un nom d'auteur (ex: l'auteur d'un fichier ou d'une fonction).  
\version  
Permet l'ajout du numéro de version (ex: dans le bloc de documentation d'un  
fichier).  
\date  
Permet l'ajout d'une date.  
\struct  
Permet la création d'un bloc de documentation pour une structure.  
\enum  
Permet la création d'un bloc de documentation pour une énumération de constantes.  
\fn  
Permet la création d'un bloc de documentation pour une fonction ou méthode.  
\def  
Permet la création d'un bloc de documentation pour une macro ou constante  
symbolique.  
\param  
Permet d'ajouter un paramètre dans un bloc de documentation d'une fonction ou d'une  
macro. Cette balise permet aussi d'ajouter en option un tag pour montrer qu'il  
s'agit d'un argument entrant, sortant ou les deux en précisant au choix: [in],  
[out]  
\return  
Permet de décrire le retour d'une fonction ou d'une méthode.  
\bug  
Permet de commencer un paragraphe décrivant une bogue (ou plusieurs).  
\class  
Permet de créer un bloc de documentation d'une classe (C++). L'utilisation est  
identique à la balise \struct  
\namespace  
Permet de créer un bloc de documentation pour un espace de nom (C++).
```

Framework GoogleTest :

Assertions Basiques

Fatal assertion	Nonfatal assertion	Vérifie
ASSERT_TRUE(<i>condition</i>);	EXPECT_TRUE(<i>condition</i>);	<i>condition</i> is true
ASSERT_FALSE(<i>condition</i>);	EXPECT_FALSE(<i>condition</i>);	<i>condition</i> is false

Comparaisons binaires (comparaisons entre deux valeurs v1 et v2 (valeurs comparables))

Fatal assertion	Nonfatal assertion	Vérifie
ASSERT_EQ(<i>expected</i> , <i>actual</i>);	EXPECT_EQ(<i>expected</i> , <i>actual</i>);	<i>expected</i> == <i>actual</i>
ASSERT_NE(<i>val1</i> , <i>val2</i>);	EXPECT_NE(<i>val1</i> , <i>val2</i>);	<i>val1</i> != <i>val2</i>
ASSERT_LT(<i>val1</i> , <i>val2</i>);	EXPECT_LT(<i>val1</i> , <i>val2</i>);	<i>val1</i> < <i>val2</i>
ASSERT_LE(<i>val1</i> , <i>val2</i>);	EXPECT_LE(<i>val1</i> , <i>val2</i>);	<i>val1</i> <= <i>val2</i>
ASSERT_GT(<i>val1</i> , <i>val2</i>);	EXPECT_GT(<i>val1</i> , <i>val2</i>);	<i>val1</i> > <i>val2</i>
ASSERT_GE(<i>val1</i> , <i>val2</i>);	EXPECT_GE(<i>val1</i> , <i>val2</i>);	<i>val1</i> >= <i>val2</i>

Assertions d'exceptions

Fatal assertion	Nonfatal assertion	Vérifie
<code>ASSERT_THROW(statement, exception_type);</code>	<code>EXPECT_THROW(statement, exception_type);</code>	<i>statement</i> throws an exception of the given type
<code>ASSERT_ANY_THROW(statement);</code>	<code>EXPECT_ANY_THROW(statement);</code>	<i>statement</i> throws an exception of any type
<code>ASSERT_NO_THROW(statement);</code>	<code>EXPECT_NO_THROW(statement);</code>	<i>statement</i> doesn't throw any exception

Test simple:

```
TEST(nom_cas_test, nom_test)
{
    ... corps de test ...
}
```

Tests utilisant un fixture, utiliser TEST_F()

Structure d'un fixture :

```
class FooTest : public ::testing::Test {
//constructeur
...
//méthodes
...
//attributs utilisés dans les tests, initialisés dans le constructeur
...
};
```

5. Barème

q1	12	4	2	2	4	
q2	6					
	6					
	11					
	9	7	2			
	12	5	1	2	4	
	29	6	5	5	6	7
q3	3	2	1			
q4	12					
	100					